O'REILLY®
Report

Compliments of

Enterprise Java Design Patterns in the Cloud Native Era

Building Resilient, Scalable Systems in the Kubernetes Era

Markus Eisele

Red Hat
Application Foundations

Turn your modernization strategy into a competitive advantage

Modernize existing applications and build new cloud-native services on a trusted, enterprise-ready platform. Red Hat Application Foundations helps you standardize runtimes, integrate systems, and accelerate your path to hybrid cloud.

redhat.com/applicationfoundations

Enterprise Java Design Patterns in the Cloud Native Era

Building Resilient, Scalable Systems in the Kubernetes Era

Markus Eisele

O'REILLY®

Enterprise Java Design Patterns in the Cloud Native Era
by Markus Eisele

Copyright © 2026 O'Reilly Media, Inc. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*https://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Andy Kwan
Development Editor: Melissa Potter
Production Editor: Aleeya Rahman
Copyeditor: Penelope Perkins
Cover Designer: Susan Thompson
Interior Designer: David Futato
Interior Illustrator: Kate Dullea

April 2026: First Edition

Revision History for the First Edition
2026-04-01: First Release

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Enterprise Java Design Patterns in the Cloud Native Era*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and Red Hat. See our statement of editorial independence.

979-8-341-67405-9

[LSI]

Table of Contents

Introduction ..... **vii**

1. Strategic Frameworks for Modernization **1**
Path 1: Maintain (Stability and Security) 2
Path 2: Migrate (Cloud Agility) 3
Path 3: Refactor (Performance and Velocity) 5
Path 4: Innovate (New Capabilities) 6

2. The Architectural Planes of Cloud Native Java **11**
Interaction and Edge Connectivity (The Front Door) 13
Execution and Communication (The Runtime) 14
Data and Intelligence (The State) 17

3. Migration and Integration Patterns **25**
The Strangler Fig Pattern 26
The Leave and Layer Pattern: Strategic Innovation 28
The Anti-Corruption Layer (ACL) Pattern: Tactical Isolation 29

4. Cloud Native Design Patterns **33**
Infrastructure Delegation Patterns: Sidecar and Ambassador 33
Data Consistency Patterns: Saga, Outbox, and Streaming 34
Decision Framework for Pattern Adoption 35

5. Architectural Approaches for Intelligent Apps **37**
Controlling AI Costs and Protecting Data 38
Making AI Understand Your Business 39
Enabling AI to Take Action 40
Where AI Fits in Your Modernization Journey 41

6. Putting It All Together..... 43
Guiding Principles 43
Where to Start 44
The Path Forward 45

Introduction

Enterprise Java systems carry real business value, but they also carry real costs. Aging monoliths slow down delivery, increase operational risk, and make it hard to adopt new capabilities like AI or cloud native infrastructure. This report gives you a practical approach to modernization: how to reduce that cost and risk while keeping the business running, and how to build systems that can grow with your organization.

Here's what we'll cover:

Strategy
: You'll get a decision-making framework for choosing the right modernization approach for each system.

Organization
: You'll learn how to organize teams and responsibilities to enable different systems to evolve at different speeds.

Migration
: You'll learn patterns for incremental modernization. No big-bang rewrites. You'll see how to modernize piece by piece and reduce risk.

Design
: You'll learn cloud native patterns for infrastructure concerns and data consistency that solve real problems.

Integration
: You'll learn how to integrate AI safely, control costs, protect data, and make probabilistic AI work in deterministic enterprise systems.

By the end, you'll have a complete toolkit for modern Enterprise Java architecture.

Acknowledgments

Writing books takes a lot more time than reading them, and it requires a lot more people to be successful at it. I am thankful to have had the technical support and creativity of Sahil Kaw, Lydwin Chinnappan, and Narang Dixita Sohanlal for this report, building on the foundation laid by Mark Little, Arun Gupta, and Daniel Bryant in *Modern Java EE Design Patterns*.

CHAPTER 1

Strategic Frameworks for Modernization

Enterprise Java has evolved from J2EE to Jakarta EE, but Kubernetes, cloud infrastructure, and generative AI demand new approaches. Traditional design patterns no longer fit modern distributed systems.

The challenge: enterprise systems now span stable OpenJDK 8 monoliths, containerized modular applications, and AI-powered microservices on GraalVM (a high-performance JVM that compiles Java to native binaries), all running simultaneously. This chapter introduces the "Four-Path Framework" for navigating this complexity and making structured modernization decisions, as shown in Figure 1-1.

Start with your main goal (stability and security, cloud agility, performance and velocity, or new capabilities) and follow the path to the appropriate modernization strategy. Each path leads to specific outcomes and technology choices that match your business objectives.

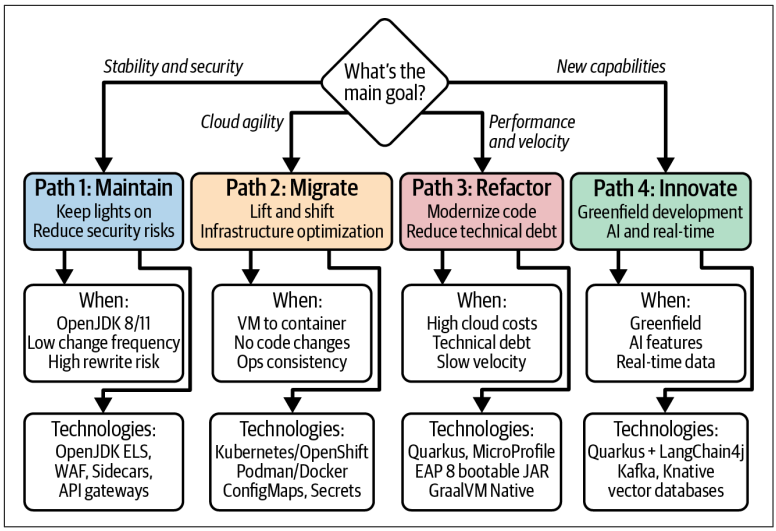

Figure 1-1. The Four-Path Framework

Path 1: Maintain (Stability and Security)

This path applies to applications that provide business value, have low change frequency, and high rewrite risk. These systems quietly run your business: processing payroll, managing inventory, or handling regulatory reporting. They work, they're stable, and nobody wants to touch them.

Let's walk through Path 1:

Goal

Keep the lights on while reducing security risks and compliance drift without code changes.

When to choose this path

- The application has no active development roadmap.
- Business users are satisfied with current functionality.
- The cost and risk of rewriting exceeds the value gained.
- The application runs on legacy Java versions (OpenJDK 8 or 11).
- You need to maintain compliance and security posture without code changes.

Optimized technology strategy

- Legacy stability: Use Extended Life Support (ELS), which is available from various vendors, for OpenJDK 8/11 to maintain security compliance for aging infrastructure.
- Edge and API defense: Implement API gateways (Kong, Apigee) and web application firewalls (ModSecurity, Cloudflare WAF, AWS WAF, Azure WAF, Envoy-based filters) for centralized authentication and virtual patching.
- Infrastructure control: Deploy service mesh (Istio, Linkerd) for granular traffic management and observability tools for proactive issue detection.
- Decision logic: If the app runs on OpenJDK 8 or 11 with no functional roadmap, use ELS. The cost of ELS is often lower than the risk-adjusted cost of a rewrite.

Path 2: Migrate (Cloud Agility)

This path focuses on infrastructure optimization without architectural refactoring, also known as "Lift and Shift." Take your existing application, containerize it, and run it on Kubernetes. You're not changing the code architecture, just where and how it runs.

Infrastructure modernization delivers real value even without code changes: better resource utilization, faster deployments, and standardized operations.

Let's go through Path 2:

Goal

Gain operational consistency by moving applications from virtual machines (VMs) to managed containers in Kubernetes.

When to choose this path

- You need to decommission expensive legacy hardware or data centers.
- You want operational consistency across all applications (legacy and modern).
- The application works well but runs on outdated infrastructure.
- You need better resource utilization and cost optimization.
- You want to enable GitOps and modern continuous integration, continuous deployment (CI/CD) pipelines.

- The business needs faster deployment cycles without code risk.
- You're consolidating multiple data centers or moving to cloud.

The migration process

- Assessment: Identify dependencies, integrations, and configuration requirements.
- Containerization: Create a Dockerfile with the right JDK version and startup command.
- Configuration externalization: Extract hardcoded values into ConfigMaps and Secrets.
- Health checks: Add HTTP endpoints for liveness and readiness probes.
- Testing: Deploy to development and verify all integrations.
- Production deployment: Use blue-green or canary strategies.

Technology choices

- Runtime and orchestration: Standardize on Kubernetes or OpenShift using Podman/Docker for secure, scalable image builds.
- Security and config: Manage environments via ConfigMaps and secure Secrets; optionally add a service mesh (Istio/Linkerd) for advanced traffic governance.
- Automation (CI/CD): Streamline deployments using Tekton, GitLab, or GitHub Actions to ensure a consistent delivery pipeline.
- Governance and visibility: Centralize image management in Quay/Docker Hub, with Prometheus/Grafana for metrics and ELK/EFK for log analysis.

Strategic value

Decommission legacy hardware, manage monoliths with the same tools as microservices, and achieve better resource utilization, faster deployments, and automated scaling.

Avoid this path if

- The application has severe performance problems requiring code-level fixes.
- You need to break down a monolith for team autonomy.

- The application uses frameworks with unpatchable security vulnerabilities.
- You're planning a complete rewrite.
- The application is tightly coupled to specific hardware, making containerization impractical.

Path 3: Refactor (Performance and Velocity)

This is the "modernize" path, which involves changing code to improve performance, reduce technical debt, or break down monoliths. The business case is driven by three factors: cloud costs (traditional Jakarta EE apps need 2–4 GB memory per instance), developer velocity (slow builds and deployments), and technical debt (tightly coupled code that's hard to test and maintain).

Here is what is important for Path 3:

Goal

Reduce cloud costs and improve developer velocity. Applications should start in seconds, use megabytes (not gigabytes), and deploy multiple times per day.

When to choose this path

- Cloud infrastructure costs are significant and growing.
- Application startup time is too slow for modern deployment practices.
- Memory footprint prevents efficient scaling.
- Developer productivity is hampered by slow build/test/deploy cycles.
- The codebase has accumulated significant technical debt.
- You need to break down a monolith for team autonomy.
- The application uses deprecated Jakarta EE features (EJB, JSF).
- You want to adopt cloud native patterns (health checks, metrics, configuration).
- The business needs faster feature delivery.

Migration strategies

- Big Bang: Rewrite everything at once (risky, only for small apps).

- Strangler Fig: Gradually replace functionality with new services (safest for large apps).
- Modular monolith first: Refactor into modules, extract services later if needed.
- Vertical slice: Migrate one complete business capability end-to-end as proof.

Technology choices

- Runtimes: Evaluate Quarkus when memory footprint, startup time, and Kubernetes-native deployment are key factors; consider prioritizing EAP 8 bootable JAR when preserving existing Jakarta EE code and minimizing migration effort. The right choice depends on your team's skills, your deployment target, and how much change the application can absorb.
- Standards: Leverage MicroProfile for microservices-focused specs (health checks, metrics, fault tolerance) or JBoss EAP 8 bootable JAR for full Jakarta EE support with traditional enterprise patterns.
- Data and API: Use Hibernate Panache to slash database boilerplate and Quarkus REST for reactive, high-throughput service layers.

Avoid this path if

- The application works fine with acceptable costs.
- You lack team capacity for multimonth refactoring.
- The business prioritizes new features over technical improvements.
- The application is scheduled for retirement within one to two years.
- Path 2 (Migrate) achieves your goals.
- The team lacks modern Java framework experience.

Path 4: Innovate (New Capabilities)

This path is for greenfield development: you're building something new without legacy constraints. Unlike the other three paths, you are starting fresh with maximum freedom to choose modern architectures, adopt the latest technologies, and optimize for the future.

The business case is driven by new revenue opportunities, competitive differentiation, or emerging capabilities.

In the age of Kubernetes, this means building modern distributed systems that can scale horizontally, handle failures gracefully, and process data in real time.

Event-driven architecture (EDA) is a strong fit when you need to decouple services, handle unpredictable loads, or process data in real time. It works well for systems where services need to react to things that happen rather than wait to be called. That said, EDA adds operational complexity: you need to manage message brokers, handle out-of-order or duplicate events, and reason about eventual consistency. It is a deliberate architectural choice, not a default. Choose it when your scalability or decoupling requirements clearly justify the overhead.

Goal

Maximum velocity and scalability. You want to move fast, experiment quickly, and scale to millions of users if the product succeeds. You need architectures that support rapid iteration and can handle unpredictable growth.

When to choose this path

- Building a new product or service from scratch
- Exploring AI-powered features that could drive new revenue
- Creating real-time experiences (live dashboards, streaming analytics, collaborative tools)
- Launching a new business line that needs its own technology stack
- Experimenting with emerging technologies before committing to them
- Building internal platforms or developer tools
- Creating customer-facing applications that need to scale rapidly
- Developing event-driven systems for complex business processes

Key benefits

- Loose coupling: Services don't need to know about each other, only about events.

- Scalability: Events are processed in parallel across multiple instances.
- Resilience: If a service is down, events queue up and get processed when it recovers.
- Audit trail: Events provide a complete history of what happened.
- Real-time processing: React to events as they happen, not in batch jobs.

Technology choices

- High-volume data streaming (Apache Kafka): Serves as the enterprise's central nervous system. It provides durable, high-throughput event logs that allow multiple services to process, replay, and analyze data in real time without impacting the source.
- Intelligent scaling (Knative and serverless): Optimizes resource costs by automatically adjusting compute power. It enables "scale-to-zero" (turning off idle services) and rapid autoscaling during traffic spikes, moving the focus from managing servers to executing code.

Avoid this path if

- You're not building something new.
- The team lacks distributed systems experience.
- The organization is not ready for microservices complexity.
- The use case doesn't require this scalability.
- You need quick delivery.
- The business case doesn't justify investment.

The decision navigator matrix in Table 1-1 provides a detailed view of the Four-Path Framework. Use this matrix to match your situation to the appropriate modernization path. Each row represents one of the four paths, showing the main goal that drives the decision, the key outcomes you can expect, and the primary technologies that enable those outcomes.

Table 1-1. Decision navigator matrix: choosing your modernization path

Path and main goal	When to choose	Key outcomes	Primary technologies	AI integration
Path 1: Maintain *Stability and Security*	• Application provides value but has no roadmap • Low change frequency • High rewrite risk • Running on OpenJDK 8/11	• Keep lights on safely • Reduce security risks • Maintain compliance • Minimal code changes	• OpenJDK ELS • API gateways (Kong, Apigee, 3scale) • WAF (ModSecurity, Cloudflare WAF) • Service mesh (Istio, Linkerd)	**AI gateway** (cost and compliance protection, no code changes)
Path 2: Migrate *Cloud Agility*	• Need operational consistency • Decommissioning legacy hardware • Want GitOps/CI/CD benefits • Application works well as is	• Infrastructure modernization • No code changes required • Operational consistency • Better resource utilization	• Kubernetes/ OpenShift • Podman/Docker • CI/CD (Tekton, GitHub Actions, GitLab) • Image registry (Quay, Docker Hub, Harbor, or Artifactory) • Observability (Prometheus/ Grafana, ELK/EFK) • ConfigMaps and Secrets	**RAG** (adds intelligence without code changes)
Path 3: Refactor *Performance and Velocity*	• High cloud costs • Slow developer velocity • Significant technical debt • Need to break down monolith	• Cost reduction • Faster startup • Improved developer velocity • Reduced technical debt	• Quarkus (REST, Hibernate Panache) • MicroProfile • EAP 8 bootable JAR • GraalVM Native Image/CRaC • OpenTelemetry • Debezium (CDC/ Outbox pattern) • CQRS (separate read/write models)	**Agentic AI** (safe tool interfaces on modern service boundaries)

Path and main goal	When to choose	Key outcomes	Primary technologies	AI integration
Path 4: Innovate *New Capabilities*	• Greenfield development • AI-powered features • Real-time data processing • New revenue streams	• Maximum velocity • Elastic scalability • AI integration • Event-driven architecture	• Quarkus + LangChain4j • Apache Kafka + Confluent Schema Registry • Knative Serverless • GraalVM Native/ CRaC • Vector databases (Qdrant, Milvus, pgvector) • OpenTelemetry • Debezium (Outbox pattern) • CQRS + event sourcing	**AI Gateway + RAG + Agentic AI + MCP** (all three patterns, designed in from the start)

The Four-Path Framework gives you a structured way to approach modernization decisions without defaulting to "rewrite everything" or "never touch it." Each path is a deliberate choice aligned to business goals: maintain what works, migrate what needs operational consistency, refactor what's costing you money and velocity, and innovate where new capabilities create competitive advantage.

In the next chapter, we'll move from strategy to execution. You'll see how these paths translate into concrete architectural patterns.

CHAPTER 2

The Architectural Planes of Cloud Native Java

Traditional enterprise architecture organized systems into horizontal tiers (presentation, business logic, data). This model breaks down in cloud native environments with dozens of services and multiple teams working in parallel. The planes model organizes architecture by concern and responsibility, not technology stack, separating platform engineering concerns from application development.

The cloud native architecture model, as shown in Figure 2-1, focuses on three runtime planes that handle how your application operates.

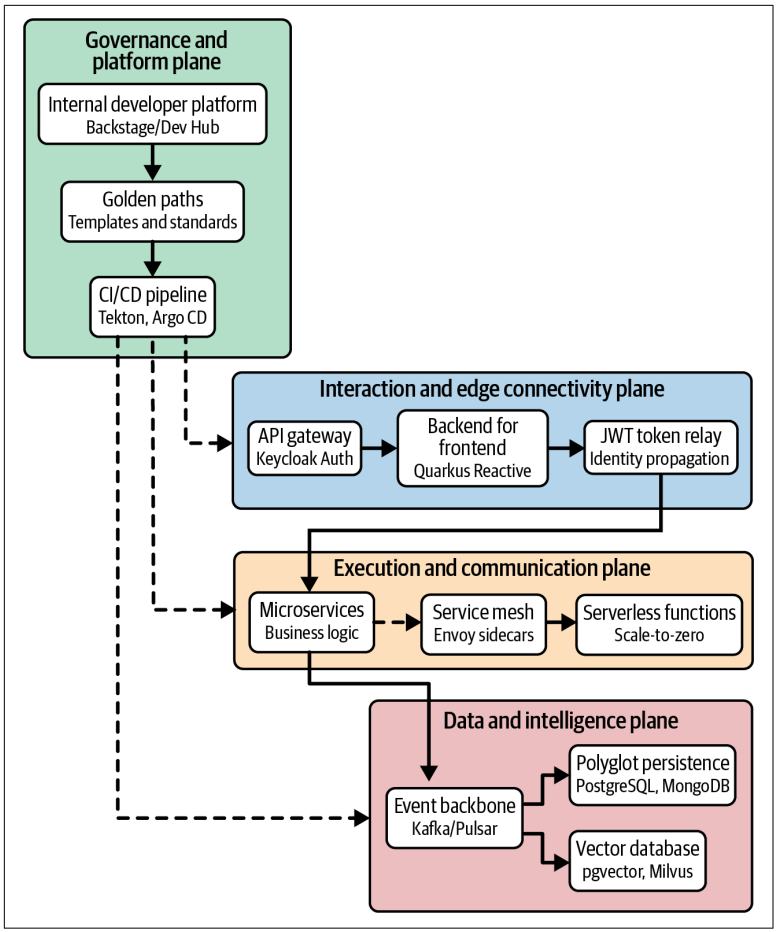

Figure 2-1. Architectural planes

Each plane is represented as a distinct layer with specific responsibilities. These planes sit on top of a platform layer that provides the foundation for deployment, monitoring, and operations. Coverage of the platform layer is beyond the scope of this report, but I recommend you read the book *Platform Engineering* by Camille Fournier and Ian Nowland (O'Reilly) for more details.

Interaction and Edge Connectivity (The Front Door)

This plane manages how traffic enters the system. In monolithic applications, a load balancer and application security constraints sufficed. In microservices architectures, the edge becomes a complex layer handling authentication, routing, rate limiting, and protocol translation across dozens of services. External clients need a stable API that doesn't change with internal refactoring.

The Edge plane solves this by providing:

- A single entry point for all external traffic
- Consistent authentication and authorization across all services
- API versioning and backward compatibility
- Rate limiting and throttling to prevent abuse
- Protocol translation (HTTP to gRPC, REST to GraphQL)
- Request aggregation to reduce client-side complexity

The most important elements of the Edge plane are:

The API gateway (the guard)
: Acts as the single entry point. It handles crosscutting concerns like rate limiting, token validation, and routing. Critically for AI, it is now the primary control point for managing large language model (LLM) token costs and enforcing prompt safety.

Backend for frontend (the concierge)
: Instead of a heavy enterprise service bus (ESB), use lightweight backends for frontends (BFFs) to tailor APIs for specific clients (e.g., stripping data for mobile versus keeping it rich for web). This prevents "over-fetching" and allows frontend teams to iterate independently.

Stateless security (the passport)
: Move away from sticky sessions. Use token relay where the gateway validates a JSON Web Token (JWT) and propagates it downstream. Services simply verify the signature, ensuring zero session state and infinite horizontal scalability.

Execution and Communication (The Runtime)

This plane ensures services communicate reliably without cluttering business logic with network code. We shift from "smart endpoint, dumb pipe" to "smart pipe, smart endpoint" while delegating retries, security, and routing to the infrastructure.

The Execution plane provides the infrastructure and patterns to make this work:

- Services can communicate without knowing the network topology.
- Failures in one service don't cascade to others.
- Traffic can be routed intelligently based on load, version, or other criteria.
- Security is enforced consistently across all service-to-service communication.
- Observability is built in, allowing you to trace requests across services and measure performance.

To build this plane, you need a set of technologies and practices that help you build services that can communicate reliably, securely, and efficiently.

Service Mesh (Sidecar) Pattern

A service mesh addresses service-to-service communication concerns (retries, security, observability) at the infrastructure level by deploying a proxy alongside each service instance, which intercepts all inbound and outbound traffic, handling those concerns transparently rather than in application code. It is a powerful tool for organizations running many services at scale, but it also adds operational complexity. Whether a service mesh makes sense for your system depends on how many services you run, how mature your platform team is, and whether the benefits outweigh the overhead of managing an additional infrastructure layer.

The sidecar proxy provides:

Automatic retries
: If a request fails, the proxy retries it automatically with exponential backoff.

Circuit breaking
: If a service is failing, the proxy stops sending requests to it temporarily.

Load balancing
: The proxy distributes requests across healthy instances.

Mutual TLS (mTLS)
: All service-to-service communication is encrypted and authenticated.

Traffic shaping
: Control timeouts, rate limits, and connection pools.

Observability
: Collect metrics, logs, and traces for every request.

Popular service mesh implementations include Istio (feature-rich with advanced capabilities), Linkerd (lightweight and simple), and Consul Connect (integrated service discovery).

When to use a service mesh:

- You have many services (10+) that need to communicate.
- You need consistent security policies across all services.
- You need advanced traffic management (canary deployments, A/B testing).
- You need detailed observability of service-to-service communication.
- You want to simplify application code by moving network logic to infrastructure.

When *not* to use a service mesh:

- You have only a few services (the operational complexity isn't worth it).
- Your services communicate primarily through message queues, not synchronous calls.
- Your team lacks the expertise to operate a service mesh.

Serverless and Scale-to-Zero

Serverless computing means you don't manage servers. You just deploy code, and the platform handles everything else. The platform automatically scales your application based on demand, even scaling to zero when there's no traffic. No instances running, no resources consumed, no costs incurred. When traffic arrives, the platform starts instances automatically.

The JVM's cold start (loading classes, just-in-time [JIT] compilation) made this impossible in the past. Traditional Java applications take seconds or even minutes to start.

Modern solutions do solve the cold start problem:

GraalVM Native Image
: Compiles Java bytecode to native machine code ahead of time. The resulting executable starts in under 100 ms and uses a fraction of the memory of a traditional JVM application.

Coordinated Restore at Checkpoint (CRaC)
: Takes a snapshot of a running JVM application and restores it instantly. The application starts in its "warmed up" state, with classes loaded and JIT compilation already done.

Knative brings serverless capabilities to Kubernetes:

Scale-to-zero
: Scale pods down to zero when idle

Autoscaling
: Automatically add pods based on request rate

Traffic splitting
: Route traffic between versions for canary deployments

Event sources
: Trigger functions from Kafka, HTTP, cron, or cloud events

Modern Java applications with fast startup and low memory usage work seamlessly with serverless platforms.

The following challenges remain:

Distributed tracing
: Debugging across multiple services is hard.

Solution: Use OpenTelemetry to trace requests across service boundaries. Each request gets a unique trace ID that follows it through all services.

Service discovery
: Services need to find each other.

  Solution: Use Kubernetes services for DNS-based discovery, or a service mesh for more advanced routing.

Cascading failures
: One slow service can bring down the entire system.

  Solution: Use circuit breakers (in the service mesh or application code) to fail fast and prevent cascading failures.

Data consistency
: Distributed transactions are hard.

  Solution: Use the Saga pattern or eventual consistency with event sourcing.

Data and Intelligence (The State)

A single shared database works well for simple applications, but it becomes a bottleneck as teams and services grow independently. Cloud native architecture often relies on polyglot persistence (using the right tool for the job) and event streaming to decouple services—though the right choice depends on your consistency requirements and operational context.

The Data and Intelligence plane addresses three fundamental challenges:

- How do services share data without sharing databases?
- How do we maintain consistency across distributed data stores?
- How do we enable real-time data processing and analytics?

The answers involve event streaming, polyglot persistence, and new data models for AI workloads.

Data is the foundation of every application. How you store, access, and process data determines your application's performance, scalability, and capabilities. The wrong data architecture can limit what you can build, no matter how good your code is.

The Data plane provides:

Data isolation
: Each service owns its data, enabling independent evolution.

Scalability
: Different data stores can scale independently based on their workload.

Real-time processing
: Event streams enable real-time analytics and reactions.

Flexibility
: Each use case utilizes the most appropriate data store, whether relational, document, graph, time-series, or vector.

Resilience
: Failures in one data store don't affect others.

From Monolithic Database to Polyglot Persistence

Polyglot persistence means using different data stores for different purposes: relational databases (PostgreSQL, MySQL) for transactional data, document databases (MongoDB, Couchbase) for semi-structured data, key-value stores (Redis, Memcached) for caching, graph databases (Neo4j) for highly connected data, time series databases (InfluxDB) for metrics, and vector databases (Qdrant, Milvus) for AI embeddings. Each service chooses the data store that best fits its needs.

This flexibility comes with real costs. Each additional data store is another system to operate, monitor, back up, and secure. Data synchronization across stores is harder to reason about, and cross-store queries or transactions require explicit design. You can adopt polyglot persistence when a specific workload has clear requirements that a single database cannot meet well, not as a default starting point. For many applications, a well-tuned relational database is still the right choice.

Eventing Backbone: The Heart of the Data Plane

Event streaming is a powerful pattern for sharing data across services without coupling them through a shared database. It works well when you need high throughput, durability, and the ability to replay history. That said, event streaming platforms come with

real operational costs: you need to manage broker clusters, monitor consumer lag, handle schema evolution, and plan for failure scenarios. Before adopting Kafka or a similar platform, make sure your team has the skills to operate it and that your workload actually justifies the infrastructure overhead. Managed services can reduce that burden significantly.

Apache Kafka is the most widely used event streaming platform. It provides:

High throughput
: Handle millions of events per second.

Durability
: Persist and replicate events across multiple brokers.

Scalability
: Add more brokers and partitions as load increases.

Retention
: Keep events for days, weeks, or forever for replay and analysis.

Ordering
: Strictly order events within a partition.

Instead of services calling each other's APIs to get data, they subscribe to events and maintain their own local copy of the data they need.

Benefits include:

Decoupling
: Services don't depend on each other's availability.

Scalability
: Each service can scale independently based on its workload.

Resilience
: If a service is down, events queue up and get processed when it recovers.

Audit trail
: Events provide a complete history of what happened.

Real-time processing
: Services react to events as they happen, not in batch jobs.

Event Sourcing: Storing State as a Sequence of Events

Event sourcing takes event-driven architecture to the next level. Instead of storing the current state of an entity, you store the sequence of events that led to that state.

Benefits include:

Complete audit trail
: Know exactly what happened and when.

Time travel
: Reconstruct state at any point in time by replaying events.

Debugging
: Reproduce bugs by replaying the exact sequence of events.

New views
: Build new read models by replaying events from the beginning.

Use event sourcing when you need a complete audit trail (financial systems, health care), you need to support time travel or undo operations, you're building event-driven systems anyway, or the complexity is justified by the benefits.

CQRS: Separating Reads from Writes

Command Query Responsibility Segregation (CQRS) separates the write model (commands that change state) from the read model (queries that retrieve data).

The main elements of CQRS are:

Write model
: Optimized for consistency and validation. Uses normalized relational database.

Read model
: Optimized for query performance. Uses denormalized views, possibly in different data stores.

Benefits include:

Performance
: Reads and writes optimize independently.

Scalability
: Read and write workloads scale separately.

Flexibility
: Diverse data stores accommodate different query patterns.

Simplicity
: Each model remains simpler by focusing on a single purpose.

Vector Data for AI: The New Frontier

AI applications, particularly those using LLMs, require a new type of data store: vector databases.

Embeddings are vectors (arrays of numbers) that represent the semantic meaning of text, images, or other data. Similar items have similar embeddings. This enables semantic search, which means finding items by meaning, not just keywords. One important pattern here is called retrieval-augmented generation (RAG). You can use Java libraries (LangChain4j) to interact with vector capabilities in PostgreSQL (pgvector) or dedicated stores like Milvus.

Use cases for vector databases include:

Semantic search
: Find documents by meaning, not keywords.

Recommendation systems
: Find similar products, articles, or users.

RAG applications
: Provide context for LLM prompts.

Duplicate detection
: Find similar or duplicate content.

Anomaly detection
: Find outliers in high-dimensional data.

Data Consistency in Distributed Systems

When data is distributed across multiple services and databases, maintaining consistency becomes challenging. You can't use traditional atomicity, consistency, isolation, and durability (ACID) transactions that span multiple databases.

Eventual consistency accepts that data might be temporarily inconsistent but will eventually become consistent. This is acceptable for

most business processes. Users don't notice a few milliseconds of inconsistency.

The Outbox pattern ensures reliable event publishing. When you update your database and publish an event, both operations must succeed or both must fail:

1. Update your business data and write the event to an "outbox" table in the same database transaction.
2. A separate process reads from the outbox table and publishes events to Kafka.
3. Mark events as published in the outbox table.

This ensures events are never lost, even if the message broker is temporarily unavailable.

Common challenges and solutions include:

Data duplication
: Each service maintains its own copy of data.

  Solution: Accept this as a trade-off for independence. Use events to keep copies synchronized.

Schema evolution
: Event schemas change over time.

  Solution: Use schema registries (Confluent Schema Registry) and design events for backward compatibility.

Event ordering
: Events might arrive out of order.

  Solution: Use Kafka partitions to guarantee ordering within a partition. Include timestamps and sequence numbers in events.

Exactly-once processing
: Ensure events are processed exactly once, not zero times or multiple times.

  Solution: Use Kafka's exactly-once semantics and idempotent consumers.

The three architectural planes organize cloud native systems by concern and responsibility, not technology layers. This separation helps you make better architectural decisions, organize teams effectively,

and identify gaps in your architecture. Development begins with the basics and adds sophistication incrementally. The planes model provides a roadmap for building maintainable, scalable systems that deliver value quickly.

The next chapter focuses on modernization patterns and approaches.

CHAPTER 3

Migration and Integration Patterns

The transition from monolith to cloud native is the most dangerous phase of modernization. To protect business continuity, you must employ patterns that enable incremental change without disrupting revenue. This chapter evaluates three strategies to address common migration scenarios: gradually replacing a monolith, extending a permanent core system, and isolating new code from legacy complexity.

Chapter 2 introduced three architectural planes as a way to organize cloud native systems by concern and responsibility. Every pattern in this report maps to one of these planes. Table 3-1 shows where each pattern lives. Use it as a reference as you read through the following chapters.

Table 3-1. Patterns mapped to architectural planes

Plane	Patterns and approaches	What they do
Interaction/ Edge	AI gateway, Ambassador	Sit between callers and services. Control routing, cost, data protection, and crosscutting concerns at the boundary.
Execution/ Runtime	Strangler Fig, Leave and Layer, Anti-Corruption Layer, Sidecar, Agentic/ MCP	Manage running workloads. Handle migration, service-to-service communication, and AI-driven automation at the runtime layer.

Plane	Patterns and approaches	What they do
Data/ Intelligence	Saga, Outbox, RAG	Address how state is stored, kept consistent, and made available. Saga and Outbox maintain data integrity across services; RAG indexes and retrieves company knowledge at runtime.

The patterns in this chapter—Strangler Fig, Leave and Layer, and Anti-Corruption Layer—all operate at the Execution/Runtime plane. They are about managing the running system during migration: shifting traffic, protecting domain models, and keeping the business running while the architecture changes underneath it.

The Strangler Fig Pattern

The Strangler Fig pattern, as shown in Figure 3-1, is the primary defense against "Big Bang" rewrites, which often fail due to duration, shifting requirements, and underestimated complexity. With this approach, traffic gradually shifts from the monolith to new services. An edge proxy routes requests based on rules you control. Change data capture (CDC) keeps databases synchronized during the transition.

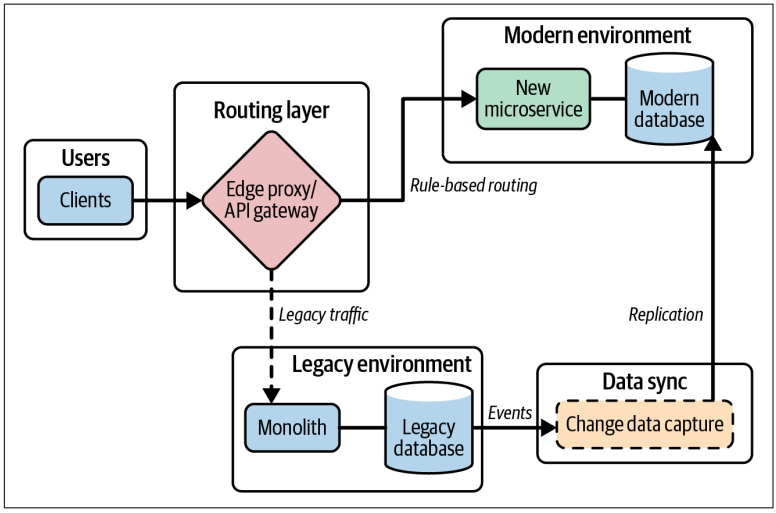

Figure 3-1. The Strangler Fig pattern

The Strangler Fig pattern delivers several key strategic advantages that make it the preferred approach for incremental modernization:

Strategic value
: Value is delivered incrementally by migrating specific bounded contexts (e.g., user profiles) one at a time.

Incremental delivered value
: You don't wait years for a complete rewrite. Each migrated capability delivers immediate benefits: faster deployments, better performance, or new features that weren't possible in the monolith.

Business continuity
: The monolith keeps running while you build the new system. Users see no disruption. Revenue continues uninterrupted.

Risk mitigation
: Traffic is shifted via an edge proxy, allowing for instant rollback if the new service fails. You prove out your new architecture with real production traffic before committing to the next migration. If something goes wrong, you can roll back instantly by changing routing rules.

Data strategy
: CDC keeps databases synchronized during the transition.

Learning as you go
: Enterprise systems encode years of business logic and edge cases. By migrating incrementally, you discover hidden dependencies and undocumented business rules before they become critical problems.

This pattern works best when:

- The monolith will eventually be retired (not a permanent fixture).
- You can identify clear, bounded contexts to migrate one at a time.
- The business needs faster feature delivery than the monolith allows.
- Cloud costs are high and you need better resource utilization.
- Developer velocity is suffering from monolith complexity.

- You have the team capacity for a multimonth migration effort.

Don't use this pattern if:

- The monolith is too risky to touch (use Leave and Layer instead).
- The application is scheduled for retirement within one to two years.
- The business needs new features more urgently than technical improvements.
- You lack the organizational commitment for incremental migration.

The Leave and Layer Pattern: Strategic Innovation

Unlike Strangler Fig, which assumes the monolith will die, Leave and Layer (as shown in Figure 3-2) assumes the monolith is permanent. In industries like banking or health care, core systems are often too risky to replace but must support modern digital channels.

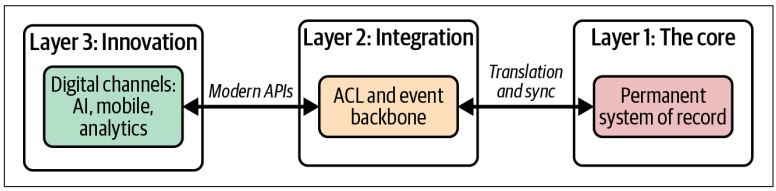

Figure 3-2. Leave and Layer pattern

This is a macro strategy that organizes your entire architecture into three distinct layers:

Layer 1 (the core)
: Remains unchanged to ensure zero risk to critical operations and regulatory compliance

Layer 2 (integration)
: Uses an Anti-Corruption Layer and CDC to translate legacy data into modern domain models

Layer 3 (innovation)
: Uses a cloud native layer for rapid feature delivery (e.g., AI, mobile apps) that scales independently of the core

Use this pattern when:

- A legacy core system is too risky to touch but needs new digital channels.
- The core system is the system of record and must remain authoritative.
- The business needs rapid innovation without waiting for core system changes.
- Regulatory or compliance requirements prevent modifying the core system.
- The core system is maintained by a different team or vendor.
- The cost and risk of replacing the core exceed the benefits.

Don't use this pattern if:

- Your organization lacks the skills to maintain both legacy and modern systems.
- The integration layer would become more complex than replacing the core.
- The legacy system cannot support the data access patterns you need (e.g., no CDC capability).

The Anti-Corruption Layer (ACL) Pattern: Tactical Isolation

The ACL is not a full-scale architecture; it is a tactical pattern used to implement migration strategies. Integrating modern services with legacy systems risks "corrupting" new domain models with messy, denormalized data structures.

Figure 3-3 shows how the ACL sits between the modern microservice and the legacy system, translating between clean domain objects and complex legacy data structures.

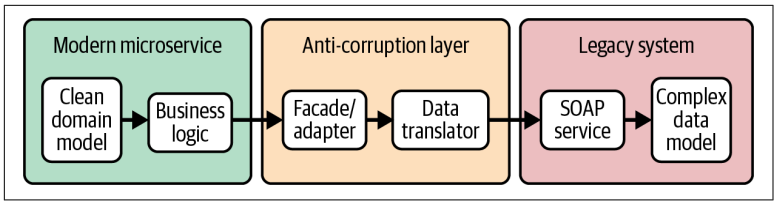

Figure 3-3. The ACL for tactical isolation

The ACL provides several key benefits that make it an essential tool in any migration strategy:

Core function
: It contains legacy complexity in one place, preventing cryptic field names and implicit rules from leaking into new code.

Strategic benefit
: When the legacy system is eventually replaced, only the ACL requires modification, leaving the core business logic untouched.

Implementation
: The ACL operates either as an "in-process" library for speed or an "out-of-process" microservice for reusability.

The ACL delivers four concrete benefits that directly improve developer productivity and system maintainability:

Protected domain model
: Your new services work with clean, well-designed domain objects that match your business concepts. The ACL handles the complexity of translating to and from legacy formats.

Isolated change impact
: When the legacy system is eventually replaced, only the ACL needs to change. Your business logic remains untouched. The ACL absorbs all the impact of replacing the legacy system.

Faster development
: Developers work with clean domain models instead of fighting legacy complexity. This improves productivity and reduces bugs.

Easier testing
: Clean domain models are easier to test. You can mock the ACL in tests without dealing with legacy system complexity.

Use an ACL pattern when:

- You're integrating with legacy systems that have complex, denormalized data models.
- The legacy system uses cryptic field names or implicit business rules.
- You want to protect your new services from legacy complexity.
- Multiple services need to integrate with the same legacy system.
- You plan to eventually replace the legacy system.

Don't use this pattern if:

- You only have one service integrating with the legacy system and no plans for more.
- The performance overhead of translation is unacceptable for your use case.
- The legacy system replacement is already budgeted and planned.
- Your team lacks the capacity to maintain the additional abstraction layer.

These three patterns form a complete toolkit for migrating from legacy systems to cloud native architectures while managing business risk.

Remember that migration is not just a technical challenge. It's an organizational challenge. You need buy-in from business stakeholders, support from operations teams, and commitment from development teams. These patterns help by reducing risk, delivering value incrementally, and providing clear success criteria.

The goal is not to migrate for the sake of migrating. The goal is to enable your organization to deliver value faster, respond to market changes more quickly, and build systems that are easier to maintain and evolve. These patterns help you achieve that goal while maintaining business continuity and managing risk.

With a solid understanding of how to migrate and integrate legacy systems, you are ready to explore the next chapter: designing the internal communication and resilience strategies that keep cloud native services reliable and responsive under real-world conditions.

CHAPTER 4

Cloud Native Design Patterns

Cloud native patterns solve distributed system problems, but each represents a trade-off between operational complexity and business capability. This chapter helps you evaluate these patterns based on ROI, risk, and implementation requirements rather than technical details.

The question is not whether these patterns work, but if your organization is ready for the operational investment they require and whether the business benefits justify the costs.

Infrastructure Delegation Patterns: Sidecar and Ambassador

The Sidecar and Ambassador patterns represent a fundamental organizational shift: crosscutting concerns move from application code to infrastructure components managed by a dedicated platform engineering team. This lets application developers focus on business logic while ensuring consistent logging, monitoring, and security across all services. Infrastructure upgrades happen independently without coordinating with application teams, reducing time spent on infrastructure concerns.

The investment requires a dedicated platform engineering team and accepts increased memory and CPU per service, more containers to monitor, and longer service startup times. Success depends on clear platform team ownership, well-defined contracts between platform and application teams, and monitoring that spans both layers.

Adopt these patterns when you have dozens of services with consistent infrastructure needs; for small teams or simple applications, the overhead exceeds the benefit. Start with noncritical services to build expertise, establish clear SLAs, and plan for platform team capacity constraints during scaling.

Data Consistency Patterns: Saga, Outbox, and Streaming

Microservices create data consistency challenges that don't exist in monoliths. Each service owns its database, eliminating single-transaction guarantees. The following patterns solve technical problems but require accepting eventual consistency and increased operational complexity.

The Saga Pattern: Coordinating Multiservice Transactions

The Saga pattern enables business processes that span multiple services without distributed transactions. Transactions take seconds or minutes instead of milliseconds, require complex compensating logic for every step, and force user experiences to accommodate temporary inconsistencies. When compensations fail, manual intervention becomes necessary. Implementation requires team expertise in distributed systems and eventual consistency, monitoring for Saga state and progress, and clear business process design that supports compensation. Target a Saga completion rate above 99%, compensation success rate above 99.9%, and average duration under 5 seconds. Invest only when you must maintain consistency across services and cannot keep related data in one service.

The Outbox Pattern: Guaranteed Event Delivery

The Outbox pattern ensures no events are lost, even during system failures. You'll need CDC infrastructure to run and monitor, accept event lag during high load (typically milliseconds, occasionally seconds), and manage storage overhead for outbox tables requiring periodic cleanup. Implementation requires CDC expertise (typically Debezium), database transaction management, and event schema design and versioning. Target an event delivery rate above 99.99% and CDC lag under 100 ms under normal load. Invest when

guaranteed event delivery is critical to business operations and you cannot afford lost events.

Data Streaming: Real-Time Integration and AI Enablement

Data streaming treats data as continuous flow rather than discrete batches, enabling both real-time application integration and AI systems that need current context. Changes propagate immediately across services (milliseconds versus hours), AI models access current data for better predictions, event-driven workflows replace polling and scheduled jobs, and you get a complete audit trail for compliance. The investment requires Kafka cluster infrastructure and operations, event schema design and versioning expertise, organizational learning curve for event-driven patterns, and more moving parts to monitor and troubleshoot.

Implementation needs streaming platform expertise (Kafka or managed alternatives), event-driven architecture skills across teams, and a data modeling and schema evolution strategy. Target event latency under 100 ms for 99th percentile, stream processing throughput that matches business load, and event schema evolution managed without breaking consumers. Invest when you need real-time data for AI systems, when applications must react to events immediately, or when integrating multiple systems without tight coupling. The complexity is not justified for simple CRUD applications or scenarios where batch processing meets business needs.

Decision Framework for Pattern Adoption

Start with patterns that address your most critical business constraints:

If feature delivery is slow due to infrastructure concerns
: Invest in Sidecar/Ambassador patterns with platform engineering team.

If you need distributed transactions
: Implement the Saga pattern, but only after confirming business processes can tolerate eventual consistency.

If event reliability is critical
: Add the Outbox pattern to ensure guaranteed delivery.

If real-time data drives business value or AI accuracy
: Invest in Data Streaming infrastructure.

Each pattern introduces its own operational complexity. You can mitigate risks through incremental adoption (start with noncritical services), team investment (ensure teams have necessary expertise before adoption), monitoring first (establish observability before deploying patterns), and escape hatches (plan for rollback if patterns don't deliver expected value).

These patterns solve real distributed system problems, but they require significant operational investment. The decision to adopt one should be driven by clear business needs and organizational readiness, not technical fashion. Ask yourself whether the pattern solves a critical business constraint, whether you have the team expertise to implement and maintain it, whether you can afford the operational overhead, and whether there's a simpler alternative that meets your needs. You can start simple and add patterns incrementally as needs grow. The goal is not to use every pattern, but to build systems that enable your organization to deliver value quickly while managing operational complexity effectively.

These patterns work best when combined with the architectural planes from Chapter 2 and migration patterns from Chapter 3. The planes provide organizational structure, migration patterns provide transition strategy, and these design patterns provide tactical implementation details for a complete cloud native approach.

In the next chapter, we'll focus on the requirements for the newly emerging, AI-infused intelligent applications.

CHAPTER 5

Architectural Approaches for Intelligent Apps

Throughout this report, we've built a foundation for modern Enterprise Java: the Four-Path Framework for strategic decisions, the architectural planes for organizing cloud native systems, and migration patterns for incremental change. Path 4 (innovate) pointed toward AI-powered features as a key driver of new capabilities. Now we look at what that actually means in practice.

Generative AI changes enterprise software fundamentally. Unlike traditional systems that produce the same output for the same input, AI models are probabilistic. The same question can lead to very different answers, and those answers can be wrong. This shift introduces challenges you must address: runaway costs, data privacy concerns, reliability issues, and the complexity of integrating AI into existing enterprise systems.

Three architectural approaches have emerged to address these challenges, each solving a specific business problem while working together to create intelligent, enterprise-grade applications.

TIP

Technical Details

For a deeper dive into implementation strategies, technical background, and the evolving landscape of modern architecture, see *Applied AI for Enterprise Java Development* by Alex Soto Bueno, Markus Eisele, and Natale Vinto (O'Reilly).

Controlling AI Costs and Protecting Data

The API gateway from the Edge plane (see Chapter 2) already handles authentication, rate limiting, and routing for all external traffic. For AI workloads, it takes on two additional responsibilities: controlling token costs and protecting sensitive data before it reaches an AI provider.

When organizations first integrate AI directly into their applications, they often encounter a painful reality: costs spiral out of control. Without proper controls, testing and production usage can quickly become expensive. Beyond cost, there's the data privacy challenge. Prompts often contain customer information, trade secrets, or personally identifiable information, and most AI providers use these prompts for training unless special agreements are in place.

The AI gateway approach solves these problems by routing all AI requests through a single, centralized service. Rather than applications calling AI models directly, they communicate with the gateway, which provides intelligent request routing, cost control, and data protection. Simple questions route to cheaper models, while complex queries go to more capable models.

The gateway scans and removes sensitive data before it reaches the AI model, tracks token usage per application or user, and enforces budgets and rate limits. Most importantly, it provides visibility. You can see who's using AI, what they're asking, and what it costs. This centralized approach prevents cost overruns and reduces compliance risk.

Because the gateway is a centralized component, it also becomes a single point of failure if not designed for resilience. To counteract this possibility, you should deploy it with redundancy, configure health checks, and define a clear failover strategy. Consider whether downstream applications should fall back to direct model access or degrade gracefully when the gateway is unavailable. Availability and failure isolation requirements should be part of your gateway design from the start, not added later.

Making AI Understand Your Business

In Chapter 2, the Data plane introduced vector databases as the storage layer for AI embeddings, and named RAG as the key pattern for providing context to LLM prompts. RAG is the runtime pattern that puts those embeddings to work.

AI models face a fundamental limitation: they don't know your company's data. They know nothing about your products, policies, or procedures. They can't access your databases, wikis, or documents and know only what they learned during training. This makes standard AI models ineffective for customer service, legal research, or developer assistance. Fine-tuning models to learn your data is expensive, slow, and impractical for information that changes frequently.

RAG solves this by injecting relevant information into prompts at runtime. The model doesn't need to know everything about your business; it just needs the right context for each specific query. Your documents, wikis, and databases are processed offline, split into chunks sized to match the embedding model's context window, and converted into vector embeddings—numerical representations that capture semantic meaning. When a user asks a question, the system searches those embeddings to find the most relevant chunks. Dense retrieval finds semantically similar content; sparse retrieval matches exact keywords; hybrid retrieval combines both for better coverage. The matching chunks are included in the prompt, allowing the AI to generate accurate, company-specific responses.

This approach works with current data without retraining, provides sources for verification to build trust, and costs less than fine-tuning. As your business information changes, you simply update the stored data rather than retrain entire models. RAG transforms generic AI into a knowledgeable assistant that understands your business context. For Java teams, LangChain4j provides the building blocks to implement this pipeline. It includes document loaders, embedding models, vector store integrations, and retrieval APIs and integrates as a native Java library that works directly with Quarkus (Spring teams can explore similar capabilities with Spring AI).

Enabling AI to Take Action

In the Leave and Layer pattern (see Chapter 3), layer 3 is the innovation layer where new digital capabilities live, separated from the stable core by an Anti-Corruption Layer. Agentic AI is what makes that layer genuinely useful. It can reach back into layer 1 and layer 2 through controlled interfaces, automating workflows that previously required manual steps across system boundaries.

Traditional AI is passive. It generates text but can't act on the world. Agentic approaches change this by allowing AI systems to call functions, query databases, and interact with external systems. A customer service agent can look up orders, process refunds, and update records. A developer assistant can search code, run tests, and create pull requests. These agents chain multiple actions together to accomplish complex tasks, turning natural language requests into automated workflows.

The Model Context Protocol (MCP) standardizes how AI agents discover and use tools. Before MCP, every AI framework had different tool formats, so building an agent for one provider meant your tools wouldn't work with another. MCP provides a universal protocol, like USB-C for AI agents, enabling any agent to use any tool. This creates reusable tools that work across different AI providers, eliminates vendor lock-in, and accelerates development by allowing teams to share tools rather than building from scratch. LangChain4j supports MCP out of the box, so Java teams can expose existing services as MCP tools and connect them to any compliant agent without rewriting integration code.

However, agents that take real actions require real safeguards. Unlike a chatbot that only generates text, an agent that can write to a database or trigger a payment needs the same trust model as a human operator. Four controls make this work in practice:

Human-in-the-loop approval
: High-risk actions—deleting records, sending emails, initiating transactions—pause and wait for explicit human confirmation before executing.

Scoped tool permissions
: Each agent declares exactly which tools it can call. An order-lookup agent has no access to the payment API, even if both tools are registered in the same MCP server.

Rate limiting and budget caps
: Agents can loop. Without limits, a misconfigured agent can exhaust an API quota or run up costs in minutes. Hard caps at the gateway level stop this before it becomes a production incident.

Immutable audit log
: Every tool call, every input, and every output is recorded with a timestamp and the identity that triggered it. This is the compliance record and the forensic trail when something goes wrong.

These controls are what separates a proof-of-concept agent from a system you can put in front of customers or regulators.

Where AI Fits in Your Modernization Journey

The three AI patterns map directly to the Four-Path Framework from Chapter 1:

Path 1 (maintain)
: The AI gateway protects stable systems from AI-related cost and compliance risk without touching the core application.

Path 2 (migrate)
: RAG adds intelligence to containerized applications without code changes to the business logic.

Path 3 (refactor)
: Agentic capabilities make sense here, where modern service boundaries provide safe, well-defined tool interfaces for agents to call.

Path 4 (innovate)
: All three patterns combine to build AI-native applications from scratch, with the AI gateway, RAG pipeline, and agent framework designed in from the start.

Start with the AI gateway regardless of which path you're on. Add RAG when your business needs AI to understand company-specific context. Add agentic capabilities last, when you have the safety controls and service boundaries to support them.

The final chapter ties everything together and helps you navigate the next steps.

CHAPTER 6

Putting It All Together

The cloud native era demands evolution from Enterprise Java architects. The skills required have shifted from memorizing specification APIs to mastering distributed system dynamics, from choosing application servers to orchestrating hybrid architectures, from writing code to curating platforms.

Guiding Principles

Three principles guide successful Enterprise Java modernization:

Focus on business value
: Start with business value, not technology—modernize because you need faster deployment, better scalability, or lower costs, not because microservices are trendy. Use the Four-Path Framework to match your goals to the appropriate strategy, and remember that sometimes the right answer is to maintain what works.

Embrace hybrid architectures as a permanent reality
: Large organizations consistently fail when they mandate a single architectural style across all systems. The reason is simple: different applications have different ages, different teams, different risk profiles, and different performance requirements. A payment processing system built in 2008 and a new AI-powered recommendation engine don't share the same constraints, and forcing them into the same architecture creates friction without adding value. Acquisitions bring foreign codebases. Regulatory

requirements freeze certain systems in place. Teams with deep domain expertise resist rewrites that would destroy years of accumulated knowledge. These are permanent conditions of operating at enterprise scale. Your enterprise will always have multiple architectural styles, so stop trying to force everything into one pattern. Instead, build platforms that support multiple approaches, use the planes model to separate concerns, and let different applications use different strategies based on their needs.

Optimize for change, not perfection
: The best architecture is one that can evolve. Build systems that are easy to change rather than systems that are perfect today. Use patterns that enable incremental improvement, avoid big-bang rewrites, make small reversible decisions, measure results, and iterate.

Where to Start

Your starting point depends on your current situation. If you're maintaining legacy systems, begin by assessing your portfolio using the Four-Path Framework and implementing API gateways to protect legacy systems from direct access. Then you can move forward by containerizing stable applications and implementing the Strangler Fig pattern for systems that need gradual replacement.

If you're building new systems, choose cloud native frameworks and implement observability from day one. Then you can add AI capabilities using the AI gateway approach and implement RAG for company-specific knowledge. Over time, you can build agentic capabilities for AI-powered automation and optimize for cost and sustainability.

If you're leading platform engineering, define golden paths for common use cases and establish self-service capabilities. Build multi-cluster management capabilities and create a platform engineering community of practice that shares knowledge and patterns across your organization.

The Path Forward

The modern Enterprise Java architect curates a hybrid estate, using the decision navigator (see Table 1-1) to choose wisely between maintaining the stability of the monolith and unleashing the velocity of the microservice. The patterns in this report work together—the Four-Path Framework guides strategy, the planes model guides organization, the migration patterns enable incremental change, the cloud native patterns solve distributed system problems, and the AI approaches integrate intelligence.

But patterns are not prescriptions. They're tools. Your job is to understand the trade-offs and apply the right pattern for your situation. Sometimes the right answer is to keep things simple. Sometimes it's to embrace complexity. The key is making informed decisions based on your specific context.

Java is not just surviving in the cloud—with these patterns, it's thriving. The platform has evolved, the ecosystem has matured, and the tools have improved. Modern Java development is faster, lighter, and more capable than ever before.

Your journey doesn't end here. Technology keeps evolving, new patterns will emerge, and AI will become more capable. But the principles remain: match architecture to requirements, embrace hybrid approaches, optimize for change, and always focus on business value.

By rejecting the dogma of one-size-fits-all and embracing the reality of hybrid architectures, you ensure that your enterprise remains resilient, agile, and ready for the next wave of innovation. The future of Enterprise Java is about orchestrating old and new to deliver business value. That's the role of the modern Enterprise Java architect.

About the Author

Markus Eisele is a principal product manager at IBM Research with over 25 years of experience helping enterprises modernize Java platforms—from monoliths to cloud native architectures with Quarkus and Kubernetes. A Java Champion and community leader, he specializes in AI-infused Java architectures, developer productivity, and platform engineering. Markus is the author of *Modernizing Enterprise Java* (O'Reilly) and *Applied AI for Enterprise Java Development* (O'Reilly). He publishes *The Main Thread*, a Substack for senior developers and architects. Follow him on Bluesky, X, and LinkedIn.